

Reservoir sampling

Reservoir sampling

Dato uno stream di elementi $x_1, x_2, x_3, \dots$ vogliamo mantenere, per ogni istante t , un insieme $R(t)$ di **esattamente** k elementi tale che $R(t)$ sia un sottoinsieme uniforme di taglia k scelto tra $\{x_1, \dots, x_t\}$.

Cosa significa “sottoinsieme uniforme di taglia k ”?

Significa:

tutti i sottoinsiemi $S \subseteq \{x_1, \dots, x_t\}$ con $|S| = k$ hanno **esattamente la stessa probabilità**

$$P\{R(t) = S\} = \frac{1}{\binom{t}{k}}.$$

In altre parole, il reservoir deve essere **indistinguibile** da un campione esatto di k elementi scelti a caso *tra i primi t* .

Reservoir sampling

Se conoscessimo in anticipo il numero totale n e avessimo memoria $\Theta(n)$:
potremmo memorizzare tutti gli n elementi, e poi estrarre k elementi scegliendo uniformemente tra tutti gli $\binom{n}{k}$ sottoinsiemi.

Questa soluzione è improponibile in molte situazioni pratiche:

- n non si conosce;
- lo stream è infinito.

Il reservoir sampling ottiene **lo stesso risultato statistico**, ma **senza conoscere n** e usando **solo memoria $O(k)$** .

Perché il problema è difficile?

La difficoltà deriva dai **vincoli strutturali**.

Non conosciamo il numero totale di elementi (lunghezza dello stream).

Non possiamo aspettare la fine per fare una scelta uniforme su $\binom{t}{k}$ possibili campioni.

Memoria limitata a $O(k)$.

Non possiamo memorizzare tutto lo stream.

Non possiamo nemmeno memorizzarne una parte “grande”: solo k elementi (con $k \ll n$).

Tempo di aggiornamento $O(1)$ per ogni nuovo elemento.

Lo stream è potenzialmente infinito e ad alta velocità.

Ogni nuovo elemento va processato con tempo e memoria costanti.

Niente strutture dati complicate, niente ricalcoli globali.

Reservoir sampling

Problema: nello streaming non possiamo salvare tutto; lo stream non termina: cresce continuamente.

Obiettivo.

Vogliamo un algoritmo che

- processi ogni elemento appena arriva,
- usi solo memoria $O(k)$,
- mantenga, ad ogni istante t , un insieme $R(t)$ di **esattamente** k elementi tale che per ogni $S \subseteq \{x_1, \dots, x_t\}$ con $|S| = k$ si abbia

$$P\{R(t) = S\} = \frac{1}{\binom{t}{k}}$$

A cosa serve?

Il reservoir sampling è usato in tutti i contesti in cui:

- I dati arrivano in streaming.
- Non si conosce la lunghezza totale.
- La memoria è limitata.
- Si vuole un *campione uniforme* del flusso.

Applicazioni tipiche

- Web analytics.
- Database e sistemi distribuiti.
- Machine learning su stream.
- Monitoraggio di rete.
- Social media / ranking contenuti.
- Finanza in tempo reale.
- Bioinformatica (sequenziamento).
- Sensor networks / IoT.
- Logging e telemetria cloud.
- E-commerce (es. Amazon).

Stream di ordini Amazon

t	ordine	categoria
1	A001	Libri
2	A002	Abbigliamento
3	A003	Abbigliamento
4	A004	Libri
5	A005	Elettronica
6	A006	Abbigliamento
7	A007	Elettronica
8	A008	Libri
9	A009	Abbigliamento
10	A010	Elettronica
11	A011	Abbigliamento
12	A012	Abbigliamento

Stream di ordini Amazon

Usiamo il reservoir sampling con $k = 4$ (*un valore molto piccolo, scelto solo per visualizzare facilmente l'output*).

Dopo aver applicato l'algoritmo, al 12esimo ordine otteniamo (in una possibile esecuzione):

$$\begin{aligned} R(12) \\ = [(A005, \text{Elettronica}), (A009, \text{Abbigliamento}), (A008, \text{Libri}), (A011, \text{Abbigliamento})] \end{aligned}$$

Categorie nel reservoir:

- Abbigliamento: $2/4$.
- Libri: $1/4$.
- Elettronica: $1/4$.

Il reservoir è un **campione uniforme di 4 ordini scelti tra i 12**.

Stream di ordini Amazon

Il reservoir è un **micro-dataset rappresentativo** degli ordini visti finora:
usa memoria $O(k)$ (qui, solo 4 ordini),
si aggiorna online in tempo $O(1)$ per ogni nuovo ordine,
è statisticamente equivalente a prendere k ordini a caso tra tutti quelli visti finora.

Da questo campione Amazon può:
stimare la **frazione di vendite per categoria**.
monitorare **trend e sbilanciamenti** in tempo reale,
alimentare modelli di **domanda** che contribuiscono (insieme a molti altri fattori) alle decisioni di **dynamic pricing**:
se la domanda per abbigliamento cresce → possibile rialzo prezzi / riduzione sconti,
se una categoria cala → possibili promozioni più aggressive.

Algoritmo Reservoir

algorithm reservoir(k)

1. alloca un array $R[1..k]$ // il reservoir
2. $t = 0$ // numero di elementi visti
3. **while** arriva un nuovo elemento x **do**
4. $t = t + 1$ // il nuovo elemento x è x_t
5. **if** $t \leq k$ **then**
6. $R[t] = x$ // x_t viene inserito alla cella t di R
7. **else**
8. // scegli una cella da aggiornare
9. $j =$ numero intero uniforme in $\{1, 2, \dots, t\}$
10. **if** $j \leq k$ **then**
11. $R[j] = x$ // x è x_t

Esempio di esecuzione

Consideriamo lo stream $\{23, 10, 12, 120, 38, \dots\}$ con $k = 3$.

Passi iniziali ($t = 1, 2, 3$).

I primi tre elementi vengono inseriti direttamente: $R(3) = (23, 10, 12)$.

Passo $t = 4$. Elemento in arrivo: $x_4 = 120$. Generiamo $j \in \{1, 2, 3, 4\}$. Supponiamo $j = 4$. Poiché $j > k$, l'elemento viene ignorato.

$$R(4) = (23, 10, 12)$$

Passo $t = 5$

Elemento in arrivo: $x_5 = 38$. Generiamo $j \in \{1, 2, 3, 4, 5\}$. Supponiamo $j = 2$. Poiché $j \leq k$, sostituiamo $R[2]$ con 38.

$$R(5) = (23, 38, 12).$$

e così via.

Efficienza e correttezza

L'efficienza dell'algoritmo di **reservoir sampling** è immediata:

- Spende tempo **$O(1)$ per elemento** dello stream.
- Usa memoria **$O(k)$** .

Vogliamo ora discutere la **correttezza**. In particolare ci interessa mostrare il seguente teorema.

Correttezza

Teorema (campione uniforme).

Dopo aver visto i primi t elementi dello stream $x_1, x_2, \dots, x_t$,

il reservoir $R(t)$ generato dall'algoritmo del reservoir sampling è un sottoinsieme **uniforme** di taglia k . Cioè:

$$P\{R(t) = S\} = \frac{1}{\binom{t}{k}} \quad \text{per ogni sottoinsieme } S \subseteq \{x_1, \dots, x_t\}, |S| = k.$$

Lemma

Lemma.

Dopo aver visto i primi t elementi, ogni elemento x_i con $i \leq t$ ha la stessa probabilità di comparire nel reservoir:

$$P\{x_i \in R(t)\} = \frac{k}{t}.$$

Dopo aver visto t elementi, ogni elemento x_i , con $i \leq t$, ha la **stessa probabilità** di essere nel reservoir:

$$P\{x_i \in R(t)\} = \frac{k}{t}.$$

In altre parole, **tutti** i primi t elementi hanno la **stessa probabilità** di comparire nel reservoir, e il reservoir rappresenta un **campione uniforme di taglia k** scelto tra $\{x_1, \dots, x_t\}$.

Dimostrazione del lemma

Per dimostrare che

$$P\{x_i \in R(t)\} = \frac{k}{t} \quad \text{per ogni } i \leq t,$$

usiamo un'**induzione su t** .

Caso base: $t = k$.

Nei primi k passi l'algoritmo inserisce direttamente gli elementi $x_1, x_2, \dots, x_k$ nel reservoir.

$$P\{x_i \in R(k)\} = 1 = \frac{k}{k}, \quad i = 1, \dots, k.$$

Dimostrazione del lemma

Passo induttivo.

Supponiamo che per qualche $t - 1 \geq k$ valga:

$$P\{x_i \in R(t - 1)\} = \frac{k}{t - 1} \quad \text{per ogni } i \leq t - 1.$$

Vogliamo dimostrare che:

$$P\{x_i \in R(t)\} = \frac{k}{t} \quad \text{per ogni } i \leq t.$$

Analizzeremo separatamente:

- caso $i < t$ (elemento già visto),
- caso $i = t$ (elemento appena arrivato).

Dimostrazione del lemma

Caso $i = t$: l'elemento è appena arrivato.

Consideriamo l'elemento **appena letto** x_t .

Al passo t , l'algoritmo genera un intero j uniforme in $\{1, 2, \dots, t\}$.

Se $j \leq k$, allora x_t viene inserito nel reservoir, sovrascrivendo la posizione j .

Se $j > k$, allora x_t viene ignorato (il reservoir resta lo stesso).

Quindi la probabilità che x_t finisca in $R(t)$ è esattamente:

$$\mathbf{P}\{x_t \in R(t)\} = \mathbf{P}\{j \leq k\} = \frac{k}{t}.$$

Il caso $i = t$ è quindi dimostrato.

Dimostrazione del lemma

Caso $i < t$: l'elemento è stato già visto.

Consideriamo un elemento x_i con $i < t$.

Vogliamo calcolare la probabilità che sia ancora nel reservoir dopo aver processato x_t .

Osserviamo che x_i può **uscire** dal reservoir *solo* se

- era in $R(t - 1)$, e
- al passo t l'algoritmo effettua un rimpiazzo (la scelta casuale cade su $j \leq k$), e
- la posizione estratta j coincide proprio con quella di x_i in $R(t - 1)$.

Dimostrazione del lemma

$$\begin{aligned} \mathbf{P}\{x_i \in R(t)\} &= \mathbf{P}\{x_i \in R(t-1) \text{ AND } x_i \text{ sopravvive al passo } t\} \\ &= \mathbf{P}\{x_i \in R(t-1)\} \cdot \mathbf{P}\{x_i \text{ sopravvive al passo } t \mid x_i \in R(t-1)\} \\ &= \frac{k}{t-1} \cdot \mathbf{P}\{x_i \text{ sopravvive al passo } t \mid x_i \in R(t-1)\} \end{aligned}$$

Ipotesi induttiva

Dobbiamo calcolare

$$\mathbf{P}\{x_i \text{ sopravvive al passo } t \mid x_i \in R(t-1)\}.$$

Dimostrazione del lemma

$$\begin{aligned} & \mathbf{P}\{x_i \text{ sopravvive al passo } t \mid x_i \in R(t-1)\} \\ &= 1 - \mathbf{P}\{x_i \text{ rimpiazzato al passo } t \mid x_i \in R(t-1)\} \end{aligned}$$

Qual è la probabilità che x_i sia rimpiazzato al successivo passo t ?

Se $x_i \in R(t-1)$, allora:

- la probabilità che ci sia un rimpiazzo è k/t ;
- condizionatamente al rimpiazzo, ognuna delle k posizioni di $R(t-1)$ è scelta con probabilità $1/k$.

Dimostrazione del lemma

$$\begin{aligned} & \mathbf{P}\{x_i \text{ sopravvive al passo } t \mid x_i \in R(t-1)\} \\ &= 1 - \mathbf{P}\{x_i \text{ rimpiazzato al passo } t \mid x_i \in R(t-1)\} \end{aligned}$$

L'evento “ x_i rimpiazzato al passo t ” è la **congiunzione** di due eventi:

Rimpiazzo(t): al passo t avviene un rimpiazzo.

Scelta(i): la posizione scelta nel rimpiazzo è proprio quella in cui si trova x_i .

Otteniamo:

$$\begin{aligned} & \mathbf{P}\{x_i \text{ rimpiazzato al passo } t \mid x_i \in R(t-1)\} \\ &= \mathbf{P}\{\text{Rimpiazzo}(t) \mid x_i \in R(t-1)\} \cdot \mathbf{P}\{\text{Scelta}(i) \mid \text{Rimpiazzo}(t), x_i \in R(t-1)\}. \end{aligned}$$

Dimostrazione del lemma

$$\begin{aligned} & \mathbf{P}\{x_i \text{ rimpiazzato al passo } t \mid x_i \in R(t-1)\} \\ &= \mathbf{P}\{\text{Rimpiazzo}(t) \mid x_i \in R(t-1)\} \cdot \mathbf{P}\{\text{Scelta}(i) \mid \text{Rimpiazzo}(t), x_i \in R(t-1)\}. \end{aligned}$$

Primo termine: probabilità di rimpiazzo.

$$\mathbf{P}\{\text{Rimpiazzo}(t) \mid x_i \in R(t-1)\} = \frac{k}{t}.$$

Questo perché l'algoritmo genera un intero uniforme $j \in \{1, \dots, t\}$ e il rimpiazzo avviene precisamente quando $j \leq k$.

Dimostrazione del lemma

$$\begin{aligned} & \mathbf{P}\{x_i \text{ rimpiazzato al passo } t \mid x_i \in R(t-1)\} \\ &= \frac{k}{t} \cdot \mathbf{P}\{\text{Scelta}(i) \mid \text{Rimpiazzo}(t), x_i \in R(t-1)\}. \end{aligned}$$

Secondo termine: probabilità di scegliere proprio la posizione di x_i .

Dato che c'è un rimpiazzo: la posizione da sostituire è scelta **uniformemente** tra le k posizioni del reservoir, e x_i occupa una sola di queste.

Dunque:

$$\mathbf{P}\{\text{Scelta}(i) \mid \text{Rimpiazzo}(t), x_i \in R(t-1)\} = \frac{1}{k}.$$

Dimostrazione del lemma

$$\begin{aligned} P\{x_i \text{ rimpiazzato al passo } t \mid x_i \in R(t-1)\} \\ = \frac{k}{t} \cdot \frac{1}{k} = \frac{1}{t} \end{aligned}$$

Quindi,

$$\begin{aligned} P\{x_i \text{ sopravvive al passo } t \mid x_i \in R(t-1)\} \\ = 1 - P\{x_i \text{ rimpiazzato al passo } t \mid x_i \in R(t-1)\} \\ = 1 - \frac{1}{t} = \frac{t-1}{t}. \end{aligned}$$

Dimostrazione del lemma

$$\begin{aligned} P\{x_i \in R(t)\} &= P\{x_i \in R(t-1) \text{ AND } x_i \text{ sopravvive al passo } t\} \\ &= P\{x_i \in R(t-1)\} \cdot P\{x_i \text{ sopravvive al passo } t \mid x_i \in R(t-1)\} \\ &= \frac{k}{t-1} \cdot P\{x_i \text{ sopravvive al passo } t \mid x_i \in R(t-1)\} \\ &= \frac{k}{t-1} \cdot \frac{t-1}{t} = \frac{k}{t}. \end{aligned}$$

Ipotesi induttiva 

Abbiamo dimostrato che, per ogni $t \geq k$ e per ogni $i \leq t$:

$$P\{x_i \in R(t)\} = \frac{k}{t}.$$

Dimostrazione del teorema

Dobbiamo dimostrare che il reservoir è un campione uniforme:

Al passo t , l'algoritmo mantiene un campione uniforme di k elementi estratti tra i primi t elementi dello stream.

Cioè dobbiamo far vedere che

Il campionamento è *equidistribuito* su tutti i sottoinsiemi di taglia k :

$$P\{R(t) = S\} = \frac{1}{\binom{t}{k}} \quad \text{per ogni } S \subseteq \{x_1, \dots, x_t\}, \quad |S| = k.$$

Dimostrazione del teorema

Dimostriamo prima che tutti i sottoinsiemi di taglia k hanno la **stessa probabilità di essere reservoir**.

Prendiamo due insiemi S, S' con $|S| = |S'| = k$.

Consideriamo uno stream arbitrario $x_1, x_2, x_3, \dots, x_t$.

Con il lemma abbiamo dimostrato che, per ogni $t \geq k$ e per ogni $i \leq t$:

$$P\{x_i \in R(t)\} = \frac{k}{t}.$$

Dimostrazione del teorema

Per ogni $t \geq k$ e per ogni $i \leq t$:

$$P\{x_i \in R(t)\} = \frac{k}{t}.$$

Scegliamo una permutazione π degli indici dello stream tale che $\pi(S) = S'$.

Questa permutazione induce uno stream permutato: $x_{\pi(1)}, x_{\pi(2)}, \dots, x_{\pi(t)}$.

- Stream originale: $x_1, x_2, \dots, x_t$.
- Stream permutato: $x_{\pi(1)}, x_{\pi(2)}, \dots, x_{\pi(t)}$.

Dimostrazione del teorema

Il lemma applicato allo stream permutato ci dice:

$$\mathbf{P}\{x_{\pi(i)} \in R(t)\} = \frac{k}{t} \quad \text{per ogni } t \geq k \text{ e per ogni } i \leq t.$$

Quindi, per ogni $t \geq k$ e per ogni $i \leq t$:

$$\mathbf{P}\{x_i \in R(t)\} = \mathbf{P}\{x_{\pi(i)} \in R(t)\} = \frac{k}{t}.$$

Dunque

$$\mathbf{P}\{R(t) = S\} = \mathbf{P}\{R(t) = S'\} = \frac{1}{\binom{t}{k}}.$$

Esame

L'esame valuta due competenze:

- Comprensione **operativa** degli algoritmi randomizzati.
- Comprensione **teorica** dei metodi probabilistici usati nello sviluppo degli algoritmi e nell'analisi.

Numero di esercizi: 5.

2 operativi + 3 teorici.

Durata complessiva: 2 ore.

Esame

Esercizi operativi (comprensione del funzionamento degli algoritmi studiati).

Obiettivo: Applicare, simulare o eseguire algoritmi randomizzati (anche solo in parte).

Esempio: Quicksort randomizzato.

Lista data.

Sequenza di pivot fissati.

Mostrare l'albero della ricorsione.

Dare l'output finale.

Punto chiave:

Comprendere come evolve l'esecuzione di un algoritmo.

Esame

Ogni esercizio operativo può contenere una o più **micro-domande teoriche a risposta breve**.

Esempio (Quicksort randomizzato):

Nell'albero della ricorsione, in quali situazioni può un nodo diventare foglia?

A cosa corrisponde la profondità dell'albero di ricorsione di Quicksort?

Perché nel caso di Quicksort il “caso medio” e il “valore atteso” coincidono?

Esame

Esercizi teorici (comprensione delle idee probabilistiche)

Obiettivo: verificare la comprensione delle idee chiave che rendono corretti ed efficienti gli algoritmi randomizzati.

Analizzano:

- concetti probabilistici (Markov, Chebyshev, Chernoff, CLT)
- meccanismi delle analisi nel caso atteso
- ragioni del successo di algoritmi Monte Carlo e Las Vegas

Esame

Tipologie di domande

Spiegare *perché* un certo evento ha una certa probabilità.

Individuare *dove* entra un teorema (es. Markov in Chebyshev).

Riassumere un passaggio chiave di una dimostrazione.

Motivare perché un algoritmo è efficiente *in atteso*.

Comprendere differenze concettuali (caso medio vs atteso, Monte Carlo vs Las Vegas).

Non serve:

- nessuna dimostrazione completa,
- nessun calcolo lungo.

Solo l'idea centrale spiegata bene.

Esame

Esempi.

Enunciare la disuguaglianza di Markov. Perché è necessario che la variabile X non assuma valori negativi?

Quali sono i passaggi chiave per dimostrare che la profondità dell'albero di ricorsione di Quicksort è $O(\log n)$ con alta probabilità?

Perché un pivot scelto uniformemente a caso è “buono” (cioè divide in due parti, entrambe di taglia $\leq 3m/4$) con probabilità $\geq 1/2$?

Esame

Balls and bins – Enunciare il Teorema del carico massimo.

Definire la v.c. B_j corrispondente al numero di palline nell'urna j . Qual è la distribuzione di B_j ? Come si calcola $E[B_j]$?

Perché per completare la dimostrazione, è sufficiente dimostrare che per un arbitrario j , $P\{B_j > k\} < \frac{1}{n^2}$?